CSE307
Principles of Programming Languages

Instructor: Zhoulai Fu

State University of New York, Korea

C crash course

- C Language Overview.
- C Environment Setup
- C Program Structure
- C Basic Syntax
- C Data Types
- C Variables
- C Constants and Literals
- C Storage Classes
- C Operators
- Decision Making in C
- C Loops
- C Functions
- C Scope Rules
- C Arrays
- C Pointers
- C Strings
- C unions

Language Overview

- Dinosaur
- Imperative.
- Close to machine

Environment Setup

- Text editor
- C compiler
- C99 (not ANSI C = C89/C90, not C11, not C17)
- gcc -std=c99 main.c
- https://www.jdoodle.com/compile-c99-online/

Program Structure

```c
#include <stdio.h>

int main()
{
   /* my first program in C */
   printf("Hello, World! \n");

   return 0;
}
```

- Preprocessor Commands
- Functions
- Variables
- Statements & Expressions
- Comments

Basic Syntax

- Program := Statements
- Statement := Tokens
- Token := Keyword | Identifier | Constant | Symbol
- Semicolon ; is statement terminator
- Comments “//…” or “/*…*/” is removed during preprocessing

Data Types

- Basic types: *int, char, float, double, long*… No bool. No string
- Void type: *void exit (int), int rand(void), void *malloc(1024)*
- enum type: *enum mbti {ESTP, INFJ…}*
- *Derived types: Pointer types, Array types, Structure types, Union types and Function types.*

Variables

- Var. declaration
- Var. definition
- Var. initialization

```
#include <stdio.h>

// Variable declaration:
extern int a, b;
extern int c;
extern float f;

int main ()
{
// Variable definition:
int a, b;
int c;
float f;

// actual initialization
  a =10;
```

declaration

Variable Declaration

- extern int x;
- Tells the compiler that these variables exist, but are defined elsewhere (in another file or later in the same file).
- No memory is allocated — it just informs the compiler of the variable's type and name.

Variable Definition

- Int x;
- Purpose: This is where the variables are actually created/ Memory is allocated.
- If a variable was declared with extern, it must eventually be defined exactly once in some translation unit.

extern is for across-file variable access

file1.c

```
int counter = 0;
// Definition: allocates storage
```

file2.c

```
extern int counter;  // Declaration: tells the
compiler it exists

void increment() {
    counter++;
// This is valid because counter is declared above
```

Question

- What does the extern keyword do in C?
- A. It creates a new variable in memory
  B. It defines a constant
  C. It declares a variable or function that is defined in another file
  D. It initializes a variable in the memory

Question

```
#include <stdio.h>
```

vs.

```
#include “stdio.h”
```

“” looks through current directory, while <> looks through system library folders

Question

- What will happen with “gcc main.c”

```c
#include <stdio.h>

extern int c;
int main()
{
    printf("%d", c);
    return 0;
}
```

Constants: Integer Literals

- decimal
- octal: 0213
- hexadecimal: 0x4b, 0xA0F
- Unsigned: 30u, 30U
- Long 42L, 42l
- Suffix is case-insensitive and can be in any order: 30ul — unsigned long

Question

- What does the following do?

- #include <stdio.h>

```
int main() {

    int num = 023;  // Octal literal

    printf("Octal 023 as decimal: %d\n", num);

    return 0;

}
```

Question

- Which one is illegal?

```
212
215u
0xFeeL
078
032UU
```

Question

- Which one is illegal?

```
212         /* Legal */
215u        /* Legal */
0xFeeL      /* Legal */
078         /* Illegal: 8 is not an octal digit */
032UU       /* Illegal: cannot repeat a suffix */
```

Question

- What does this program produce?

```c
#include<stdio.h>

int main(void) {
    int x=077u;
    int y=0xfeel;
    int z=x+y;
    printf("x = %i\n", x);
    printf("y = %i\n", y);
    printf("Sum of x+y = %i\n", z);
}
```

Try this: jdoodle.com/ia/IB5

Question

- What does the following do?

```
#include<stdio.h>
int main(void) {
    int a = -1;
    unsigned int b = 077u;
    if (a > b) {
        printf("a is greater\n");
    } else {
        printf("b is greater\n”); }
```

Constants: Floating-point literals

- Decimal form
- Scientific notation form

```
3.14159        /* Legal */
314159E-5L     /* Legal */
510E           /* Illegal: incomplete exponent */
210f           /* Illegal: no decimal or exponent */
.e55           /* Illegal: missing integer or fraction */
```

Question

- What does this program produce?

```c
#include<stdio.h>

int main()
{

    if (0.1 + 0.2 == 0.3 )
        printf ("Yes. 0.1 + 0.2 == 0.3 \n");
    else
        printf ("No. 0.1 + 0.2 != 0.3 \n");


    return 0;
}
```

jdoodle.com/ia/IB9

Constants: chars

- plain character (e.g., 'x'),
- escape sequence (e.g., ‘\t')
- universal character (e.g., '\u02C0').

Escape sequence	Meaning
\\	\ character
\'	' character
\"	" character
\?	? character
\a	Alert or bell
\b	Backspace
\f	Form feed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\ooo	Octal number of one to three digits

Try this

```c
#include <stdio.h>

int main() {
    printf("1. Hello, World!\n");          // newline escape character
    printf("2. Hello,\tWorld!\n");         // tab escape character
    printf("3. Hello,\\World!\n");         // backslash escape character
    printf("4. Hello,\'World!\n");         // single quote escape character
    printf("5. Hello,\"World!\n");         // double quote escape character
    printf("6. Hello,\aWorld!\n");         // alert(bell) escape character
    printf("7. Hello,\bWorld!\n");         // backspace escape character
    printf("8. Hello,\fWorld!\n");         // form feed escape character
    printf("9. Hello,\rWorld!\n");         // carriage return escape character
    printf("10. Hello,\vWorld!\n");         // vertical tab escape character
    printf("11. Hello,\x48World!\n");       // hexadecimal number escape character
    printf("12. Hello,\101World!\n");       // octal number escape character
    printf("13. Hello,\u03B1World!\n");     // unicode escape character

    return 0;
}
```

```c
#include <stdio.h>

int main() {
    printf("1. Hello, World!\n");
    printf("2. Hello,\tWorld!\n");
    printf("3. Hello,\\World!\n");
    printf("4. Hello,\'World!\n");
    printf("5. Hello,\"World!\n");
    printf("6. Hello,\aWorld!\n");
    printf("7. Hello,\bWorld!\n");
    printf("8. Hello,\fWorld!\n");
    printf("9. Hello,\rWorld!\n");
    printf("10. Hello,\vWorld!\n");
    printf("11. Hello,\x48World!\n");
character
    printf("12. Hello,\101World!\n");
    printf("13. Hello,\u03B1World!\n");

    return 0;
}
```

```
1. Hello, World!
2. Hello,       World!
3. Hello,\World!
4. Hello,'World!
5. Hello,"World!
6. Hello,World!
7. HelloWorld!
8. Hello,
          World!
World!lo,
10. Hello,
           World!
11. Hello,HWorld!
12. Hello,AWorld!
13. Hello,αWorld!
```

Constants: strings

- strings = char sequences ending with \0
- break a long line into multiple lines = separate them using whitespaces
- All the three forms are identical

```
"hello, dear"

"hello, \

dear"

"hello, " "d" "ear"
```

Defining Constants

Using #define preprocessor.

```c
#include <stdio.h>

#define LENGTH 10
#define WIDTH  5
#define NEWLINE '\n'

int main()
{

   int area;

   area = LENGTH * WIDTH;
   printf("value of area : %d", area);
   printf("%c", NEWLINE);

   return 0;
}
```

Using const keyword.

```c
#include <stdio.h>

int main()
{
   const int  LENGTH = 10;
   const int  WIDTH  = 5;
   const char NEWLINE = '\n';
   int area;

   area = LENGTH * WIDTH;
   printf("value of area : %d", area);
   printf("%c", NEWLINE);

   return 0;
}
```

#define LENGTH vs const int LENGTH ?

Storage classes

- auto:
- register:
- static:
- extern:

Storage classes

- auto: Variable allocated when the block in which they are defined is entered, and deallocated when it is exited.
- register: local variables that should be stored in a register instead of RAM.
- static: existence during the life-time
- extern: give a reference of a global variable that is visible to ALL the program files.

Auto

- auto: Variable allocated when the block in which they are defined is entered, and deallocated when it is exited.

```
void function() {
    auto int x = 0; // Here, `auto` is redundant because `x` is a local variable
    // ...
}
```

Register

- register: local variables that should be stored in a register instead of RAM.

```c
#include <stdio.h>

int main() {
    register int counter;
    for(counter=0; counter<100000; counter++) {
        printf("%d\n", counter);
    }
    return 0;
}
```

Static

- static: existence during the life-time

```c
#include <stdio.h>

void increment() {
    static int count = 0;
    count++;
    printf("%d\n", count);
}

int main() {
    increment();  // prints 1
    increment();  // prints 2
    increment();  // prints 3
    return 0;
}
```

extern

- extern: give a reference of a global variable that is visible to ALL the program files.

First File: main.c

```c
#include <stdio.h>

int count ;
extern void write_extern();

main()
{
   write_extern();
}
```

Second File: write.c

```c
#include <stdio.h>

extern int count;

void write_extern(void)
{
   count = 5;
   printf("count is %d\n", count);
}
```

Lab exercise 1:
Implementing a Caesar Cipher in C

Introduction

- In this lab exercise, you will be implementing a simple Caesar cipher in the C programming language. A Caesar cipher is a type of substitution cipher where each character in the plaintext is 'shifted' a certain number of places down the alphabet. For example, with a shift of 3, A would be replaced by D, B would become E, and so on.

Background

- **Character arrays in C:** In C, strings are typically represented as arrays of characters. For example, the string "HELLO" can be declared as **char str[] = "HELLO";**. Note that all strings in C are null-terminated, which means they end with a special character '\0'.

- **Character pointers in C (char*)**: A character pointer in C can also be used to represent a string. It can point to the first character of a string, and the string is assumed to continue until a null character is encountered. For example, **char* str = "HELLO";**.

- **String manipulation in C**: C provides several functions for manipulating strings, such as **strcpy** for copying strings and **strlen** for finding the length of a string. However, in this exercise, you will be manipulating strings directly.

Problem Statement

```c
int main() {
    char str[] = "KENNEDY";
    caesarCipher(str);
    printf("%s\n", str); // Should print "NHQQHGB"
    return 0;
}
```

- Write a C function **void caesarCipher(char* str)** that performs a Caesar cipher on an input string. The string will consist of capital letters only, and the cipher should shift each letter 3 places to the right in the alphabet, wrapping around to the beginning of the alphabet if necessary.

- For example, the input string "KENNEDY" should produce the output "NHQQHGB".

Lab exercise 2:
Sentence Title Case Verification in C

Problem

- Your task is to write a C function that checks whether a sentence is in 'Title Case'. In other words, the function should return true if each word in the sentence starts with a capital letter and continues with lowercase letters. Here are the specific requirements:

    - The function should take a single argument - a string, representing the sentence to check. This string consists only of letters and blank spaces.

    - The function should return a boolean value (in C, typically represented as an int with 0 for false and non-zero for true).

    - The function should return true if and only if each word in the sentence starts with a capital letter and continues with lowercase letters. Otherwise, it should return false.

- Write the function as described above. Test your function with several test sentences to ensure that it works correctly.

Background

- In C, strings are represented as arrays of characters. You can use array indexing to access individual characters in a string, similar to how you'd access elements in an array. For example, sentence[0] would give you the first character in the string sentence.

- C provides functions to manipulate and check characters. You might find the following functions from the ctype.h library useful:

  - **isupper(int c)** checks if the given character is uppercase.

  - **islower(int c)** checks if the given character is lowercase.

  - **isspace(int c)** checks if the given character is a whitespace character.

- Reminder: A string in C is null-terminated, meaning it ends with the special null character '\0'. You can use this fact to iterate through the string.

C crash course (cont.)

In the following

- **C Pointers**
- **Lab questions**

FANG Interview Question 1: const char* vs. char* const

- What does **const char* s = “hello";** do ?

- **const char*** is a pointer to a "constant character”. This means you're not allowed to change the character that the pointer is referring to

FANG Interview Question 1: const char* vs. char* const

- What does **char* const str = “hello";** do ?

- **char* const** is a “constant pointer” to a character. This means you're not allowed to change the pointer

FANG Interview Question 2: What is void*?

- generic pointer type
- raw address in memory
- can store the address of any object
- can be type-casted to any type of pointer
- Cannot be directly dereferenced. To access the object, you must first cast void * to a correct pointer type.

```c
#include <stdio.h>

void printNumber(void *ptr, char type) {
    if (type == 'i') { // If the type is integer
        int *int_ptr = (int *)ptr;
        printf("The number is %d\n", *int_ptr);
    } else if (type == 'f') { // If the type is float
        float *float_ptr = (float *)ptr;
        printf("The number is %f\n", *float_ptr);
    }
}

int main() {
    int i = 5;
    float f = 9.5;

    // Passing integer pointer
    printNumber(&i, 'i');

    // Passing float pointer
    printNumber(&f, 'f');

    return 0;
}
```

Address of (&)

- To get the address of a variable, we use the ampersand (&) operator,

```c
#include <stdio.h>

int main() {
    int var = 5;
    printf("Address of var: %p\n", (void*)&var);
    return 0;
}
```

Pointers (*)

- A pointer is a variable whose value is an address
- To define a pointer, use **type *var-name;**

```
int    *ip;    /* pointer to an integer */
double *dp;    /* pointer to a double */
float  *fp;    /* pointer to a float */
char   *ch     /* pointer to a character */
```

Question

- If the 1st printf gives 0x7fff5c7ea38c, what will be the results of the 2nd and 3rd printf?

```c
#include <stdio.h>
int main () {
   int var = 20;
   int *ip = &var;

   printf("Address of var variable: %p\n", (void*) &var  );

  printf("Address stored in ip variable: %p\n", (void *) ip );

  printf("Value of *ip variable: %d\n", *ip );
  return 0;
}
```

Question

- Any difference between int *ptr and int* ptr, or int * ptr?

- What is the type of “b” in **int* a, b**?

- What is the type of “b” in **int *a, *b**?

NULL Pointer

- The NULL pointer always points to no objects
- int *ptr = NULL;
- cannot be dereferenced

Issue from an online tutorial

The **NULL** pointer is a constant with a value of zero defined in several standard libraries. Consider the following program:

```
#include <stdio.h>

int main ()
{
   int  *ptr = NULL;

   printf("The value of ptr is : %x\n", &ptr  );

   return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
The value of ptr is 0
```

Question: What will happen if we run this

```c
#include <stdio.h>
int main () {

  int *ptr = NULL;

  printf("The value of ptr is : %p\n", (void *) &ptr  );

  printf("The value of ptr is : %p\n", (void *) ptr  );

  printf("The value of ptr is : %x\n", *ptr  );

return 0;
}
```

Try: jdoodle.com/ia/IN3

Answer

```c
#include <stdio.h>
int main () {

  int *ptr = NULL;

  printf("The value of ptr is : %p\n", (void *) &ptr  );

  printf("The value of ptr is : %p\n", (void *) ptr  );

  printf("The value of ptr is : %x\n", *ptr  );

return 0;
}
```

:

The value of ptr is : 0x7ffc5aa40fb0

The value of ptr is : (nil)

Segmentation fault (core dumped)

Bytecode for java helloworld

```
00000000: CA FE BA BE 00 00 00 32|00 1D 0A 00 06 00 0F 09 |
00000010: 00 10 00 11 08 00 12 0A|00 13 00 14 07 00 15 07 |
00000020: 00 16 01 00 06 3C 69 6E|69 74 3E 01 00 03 28 29 |
00000030: 56 01 00 04 43 6F 64 65|01 00 0F 4C 69 6E 65 4E |
00000040: 75 6D 62 65 72 54 61 62|6C 65 01 00 04 6D 61 69 |
00000050: 6E 01 00 16 28 5B 4C 6A|61 76 61 2F 6C 61 6E 67 |
00000060: 2F 53 74 72 69 6E 67 3B|29 56 01 00 0A 53 6F 75 |
00000070: 72 63 65 46 69 6C 65 01|00 0F 48 65 6C 6C 6F 57 |
00000080: 6F 72 6C 64 2E 6A 61 76|61 0C 00 07 00 08 07 00 |
00000090: 17 0C 00 18 00 19 01 00|0B 48 65 6C 6C 6F 20 57 |
000000A0: 6F 72 6C 64 07 00 1A 0C|00 1B 00 1C 01 00 0A 48 |
000000B0: 65 6C 6C 6F 57 6F 72 6C|64 01 00 10 6A 61 76 61 |
000000C0: 2F 6C 61 6E 67 2F 4F 62|6A 65 63 74 01 00 10 6A |
000000D0: 61 76 61 2F 6C 61 6E 67|2F 53 79 73 74 65 6D 01 |
000000E0: 00 03 6F 75 74 01 00 15|4C 6A 61 76 61 2F 69 6F |
000000F0: 2F 50 72 69 6E 74 53 74|72 65 61 6D 3B 01 00 13 |
00000100: 6A 61 76 61 2F 69 6F 2F|50 72 69 6E 74 53 74 72 |
00000110: 65 61 6D 01 00 07 70 72|69 6E 74 6C 6E 01 00 15 |
00000120: 28 4C 6A 61 76 61 2F 6C|61 6E 67 2F 53 74 72 69 |
00000130: 6E 67 3B 29 56 00 20 00|05 00 06 00 00 00 00 00 |
00000140: 02 00 00 00 07 00 08 00|01 00 09 00 00 00 1D 00 |
00000150: 01 00 01 00 00 00 05 2A|B7 00 01 B1 00 00 00 01 |
00000160: 00 0A 00 00 00 06 00 01|00 00 00 02 00 09 00 0B |
00000170: 00 0C 00 01 00 09 00 00|00 25 00 02 00 01 00 00 |
00000180: 00 09 B2 00 02 12 03 B6|00 04 B1 00 00 00 01 00 |
00000190: 0A 00 00 00 0A 00 02 00|00 00 06 00 08 00 07 00 |
000001A0: 01 00 0D 00 00 00 02 00|0E                     |
```

Check if a pointer is Null

```
if(ptr)     /* succeeds if p is not null */
if(!ptr)    /* succeeds if p is null */
```

FANG Interview Question 3: Pointer arithmetics

- Assume ptr is an 32-bit integer pointer which points to the address 0x1009.

- ptr++ = ?

- Assume ptr is a char pointer which points to the address 1009.

- ptr++ = ?

Arrays

- Arrays are constant pointers
- int x[5];
- x++ would not work
- Use int * ptr = x; ptr++;

FANG Interview Question 4: Which one will crash, and why?

```c
#include <stdio.h>

const int MAX = 3;
int main () {

    int var[] = {10, 100, 200};

    double b[5] = {1000.0, 2.0, 3.4, 17.0, 50.0};
    double *ptr=b;

    for (int i = 0; i < 5; i++) {
        printf("Value of balance[%d] = %f\n", i, *ptr );
        /* move to the next location */
        ptr++;
    }
    return 0;
}
```

```c
#include <stdio.h>

const int MAX = 3;
int main () {

    int var[] = {10, 100, 200};

    double b[5] = {1000.0, 2.0, 3.4, 17.0, 50.0};

    for (int i = 0; i < 5; i++) {
        printf("Value of b[%d] = %f\n", i, *b );
        /* move to the next location */
        b++;
    }
    return 0;
}
```

C Exercises

1

- What will be the output of the following C code?

```
int x = 10;

int *p = &x;

*p = 20;

printf("%d", x);
```

2

- What will be the output of the following C code?

```
#include <stdio.h>

void trickyFunction(int *p1, int *p2) {
    *p1 = *p1 + *p2;
    p1 = p2;
    *p1 = *p1 - *p2;
}

int main() {
    int x = 5, y = 10;
    int *ptr1 = &x, *ptr2 = &y;

    trickyFunction(ptr1, ptr2);

    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

1. `x = 15, y = 0`
2. `x = 15, y = 10`
3. `x = 15, y = -10`
4. Compilation error.

3

- What will be the output of the following C code?

```
int main() {

    int arr[] = {10, 20, 30, 40, 50, 60};

    int *ptr = arr;

    printf("%d ", *(ptr++));

    printf("%d", *ptr);

    return 0;

}
```

Important parts about C

- Can write simple C code involving strings and pointers
- Low-level data representation like bytes
- Pointer manipulation, alias issues